HO CHI MINH UNIVERSITY OF BANKING

Enhancing Computational Reliability of AI Systems through MCP Safe Calculation Math Engine

APPENDICES

This volume collects the appendices of the master's capstone project and is published as a separate companion volume.

Author:	Lam Thanh Phong
Supervisor:	Dr. Nguyen Hoai Duc

Ho Chi Minh University of Banking

2026

CONTENTS OF THE APPENDIX VOLUME

A SCME Source Code **A-4**

A.1 MCP Server Structure (TypeScript) A-4

A.2 Python Calculation Core (CLI) A-4

A.3 MCP Tool Usage Examples A-6

B Detailed Test Result Tables **A-7**

B.1 Basic Test Cases A-7

B.2 Advanced Test Cases A-8

C Installation Guide **A-9**

C.1 System Requirements A-9

C.2 Installation from Source A-9

C.3 Configuration for VSCode/Cursor A-9

C.4 Configuration for Claude Code A-10

C.5 Operational Check A-10

C.6 System Dependencies A-11

D SCME vs GPT-4o-mini Comparison **A-12**

D.1 API Call Information A-12

D.2 Comparison of TikZ Computation Results A-12

D.3 Difference Analysis A-13

D.3.1 Main Differences A-13

D.3.2 Transparency Notes A-14

D.3.3 Experimental Limitations A-14

E Evidence archive **A-15**

F Application proposals **A-17**

F.0.1 Real-world applications of SCME A-17

F.0.2 Applications in Finance and Banking A-17

F.0.3 Deployment recommendations A-18

G State of research: comparison tables **A-19**

G.1 LLM progress in mathematics (2023–2025) A-20

H IDE integration **A-23**

I IEEE 754 Number Representation Details **A-24**

J Scope, baselines and measurement limits **A-25**

J.0.1 Scope, baselines and measurement limits A-25

K Cost–benefit analysis **A-29**

K.0.1 Cost–benefit analysis A-29

L Recommendations **A-31**

L.0.1 Recommendations A-31

M Visual Illustrations of Calculation Errors **A-33**

M.0.1 Visual Illustrations of Calculation Errors A-33

N Architecture of large language models **A-34**

N.0.1 Architecture of large language models (LLM) A-34

O Hallucination in mathematical computation **A-36**

O.0.1 Hallucination in mathematical computation A-36

P Background Tables **A-38**

P.1 Representative LLMs (2023 to 2024) A-38

P.2 LLM market milestones, 2020 to 2026 . . . . . A-38

P.3 Chapter 3 summary (recap moved from the main volume) A-39

P.4 Reliability methods: background discussion A-40

Q Root Causes of LLM Calculation Errors **A-41**

Q.1 Five Whys Analysis A-41
Q.2 Summary of Causes A-41

APPENDIX A
SCME SOURCE CODE

A.1 MCP Server Structure (TypeScript)

```typescript
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from
  "@modelcontextprotocol/sdk/server/stdio.js";
import { CallToolRequestSchema } from
  "@modelcontextprotocol/sdk/types.js";

const server = new Server({
  name: "safe-calculation",
  version: "1.1.0",
}, {
  capabilities: { tools: {} }
});

server.setRequestHandler(CallToolRequestSchema, async (request) => {
  const { name, arguments: args } = request.params;
  if (name === "calculate") {
    return await executeCalculation(args);
  }
  throw new Error(`Unknown tool: ${name}`);
});

const transport = new StdioServerTransport();
await server.connect(transport);
```

Listing A.1: index.ts - MCP Server Entry Point

A.2 Python Calculation Core (CLI)

```python
from sympy import *
from sympy.parsing.sympy_parser import parse_expr
import json

def eval_expression(expression: str, precision: int = 15) -> dict:
    try:
        expr = parse_expr(expression)
        result = expr.evalf(precision)
```

```
        return {
            "status": "success",
            "result": str(result),
            "symbolic": str(expr)
        }
    except Exception as e:
        return {"status": "error", "message": str(e)}

def stats_describe(data: list) -> dict:
    import numpy as np
    arr = np.array(data)
    return {
        "count": len(arr),
        "mean": float(np.mean(arr)),
        "median": float(np.median(arr)),
        "std": float(np.std(arr, ddof=1)),
        "variance": float(np.var(arr, ddof=1)),
        "min": float(np.min(arr)),
        "max": float(np.max(arr)),
        "q1": float(np.percentile(arr, 25)),
        "q3": float(np.percentile(arr, 75))
    }
```

Listing A.2: core.py - Calculation Engine Core

A.3 MCP Tool Usage Examples

```
// 1. Eval
await mcp__safe_calculation__calculate({
  operation: "eval",
  params: { expression: "sqrt(144) + sin(pi/6)" }
});

// 2. Batch
await mcp__safe_calculation__calculate({
  operation: "batch",
  params: {
    operations: JSON.stringify([
      { name: "sum", operation: "eval",
        params: { expression: "100+200+300" }},
      { name: "avg", operation: "eval",
        params: { expression: "(100+200+300)/3" }}
    ])
  }
});

// 3. Stats
await mcp__safe_calculation__calculate({
  operation: "stats_describe",
  params: { data: "[85,90,78,92,88]" }
});
```

Listing A.3: Example MCP calls from an LLM

APPENDIX B
DETAILED TEST RESULT TABLES

B.1 Basic Test Cases

Table B.1: Details of arithmetic and trigonometric test cases

ID	Expression	Expected	Actual	Result
T01	$2+3$	5	5	Pass
T02	7×8	56	56	Pass
T03	$\sqrt{144}$	12	12	Pass
T04	2^{10}	1024	1024	Pass
T05	$\sqrt{2}$	1.4142	1.4142135623730951	Pass
T06	$\sin(\pi/6)$	0.5	0.5	Pass
T07	$\cos(\pi/3)$	0.5	0.5	Pass
T08	$\tan(\pi/4)$	1	1	Pass
T09	$\ln(e)$	1	1	Pass
T10	factorial(5)	120	120	Pass

Source: author's experiment (evidence item E1), selected basic cases (all rows are stored cases of that evidence item).

B.2 Advanced Test Cases

Table B.2: Details of statistical and complex-number test cases

ID	Operation	Expected	Actual	Result
T11	mean([85, 90, 78, 92, 88, 76, 95, 82, 89, 91])	86.6	86.6	Pass
T12	median([85, 90, 78, 92, 88, 76, 95, 82, 89, 91])	88.5	88.5	Pass
T13	stdev([85, 90, 78, 92, 88, 76, 95, 82, 89, 91])	6.22	6.22	Pass
T14	$(2+3i)+(4+5i)$	$6+8i$	$6+8i$	Pass
T15	$(2+3i)\times(1-4i)$	$14-5i$	$14-5i$	Pass
T16	100 km $\rightarrow$ miles	62.137	62.137	Pass
T17	32°F $\rightarrow$ Celsius	0	0	Pass
T18	Linear regression R^2	0.6	0.6	Pass

Source: author's experiment (evidence item E1), selected advanced cases (all rows are stored cases of that evidence item).

APPENDIX C
INSTALLATION GUIDE

C.1 System Requirements

- Node.js version 20.0.0 or later
- Python 3.10 or later
- Git for cloning the repository
- An MCP-capable IDE: VSCode, Cursor, or Claude Code

C.2 Installation from Source

```
# 1. Clone the public release v1.1.0
git clone --branch v1.1.0 \
  https://github.com/limpaulfin/scme-safe-calculation-math-engine.git
     scme && cd scme

# 2. Install MCP Server (TypeScript)
cd mcp
npm install
npm run build

# 3. Install Python dependencies
cd ../mcp-safe-calculation-math-engine-core
python3 -m venv venv
source venv/bin/activate  # Linux/Mac
pip install -r requirements.txt

# 4. Verify installation
node mcp/dist/index.js --version
python core.py --test
```

Listing C.1: SCME installation workflow

C.3 Configuration for VSCode/Cursor

```
{
  "mcpServers": {
    "safe-calculation": {
      "command": "node",
      "args": ["/path/to/mcp/dist/index.js"],
      "cwd": "/path/to/mcp-safe-calculation",
      "description": "SCME - 27 operations"
    }
  }
}
```

Listing C.2: MCP configuration in settings.json

C.4 Configuration for Claude Code

```
# File: ~/.claude/settings.json
{
  "mcpServers": {
    "safe-calculation": {
      "command": "node",
      "args": ["/absolute/path/to/mcp/dist/index.js"],
      "cwd": "/absolute/path/to/mcp"
    }
  }
}

# Verify
claude --mcp-servers
# Output should include: safe-calculation
```

Listing C.3: Claude Code configuration

C.5 Operational Check

After configuration, restart the IDE and run the following checks:

1. Open the terminal/chat in the IDE
2. Ask the AI to compute: “Calculate the square root of 144”
3. The AI automatically invokes the MCP tool and returns the result: 12
4. Verify with more complex computations: statistics, complex numbers, regression

C.6 System Dependencies

Table C.1 lists the software dependencies of the SCME system by layer.

Table C.1: Dependencies of the SCME system

Layer	Library	Purpose
MCP Server	@modelcontextprotocol/sdk	MCP protocol
MCP Server	execa	Process execution
MCP Server	zod	Schema validation
Python Core	sympy	Symbolic math
Python Core	scipy	Scientific computing
Python Core	numpy	Numerical arithmetic
Python Core	pint	Unit conversion

Source: compiled by the author from the dependency set of the SCME engine release.

APPENDIX D
SCME VS GPT-4O-MINI COMPARISON

This appendix presents empirical comparison results between SCME and the GPT-4o-mini API for computing TikZ coordinates. Data were collected directly from API calls; Table D.1 reports the call metadata used to ensure transparency and reproducibility of the study.

D.1 API Call Information

Table D.1: Metadata of the compared API calls

Attribute	**SCME**	**GPT-4o-mini**
Experiment ID	TIKZ-COORD-2025-12-24-005718	
Tool/API	mcp__safe-calculation	OpenAI Chat Completions
Model/Tool	SymPy + NumPy	gpt-4o-mini
Timestamp	2025-12-24 00:56:45 +07	2025-12-24 00:57:18 +07
Duration	$\sim$200ms (batch of 18 ops; local function call)	8.49s (token generation; not like-for-like, see the note below)
Script	MCP Protocol (stdio)	OpenAI API shell script

Source: author's experiment (evidence item E9).

D.2 Comparison of TikZ Computation Results

Table D.2 compares the computation of 18 trigonometric expressions for three TikZ examples: (1) a 45° rotation of point $P(3,0)$, (2) the coordinates of six vertices of a regular hexagon with $r = 2$, and (3) a 30° rotation matrix transformation of point $P(2,0)$.

Table D.2: Comparison of computation results: SCME vs GPT-4o-mini

ID	Expression	SCME (symbolic)	GPT-4o-mini	Deviation
1	$3\cos(45^\circ)$	$\frac{3\sqrt{2}}{2} = 2.1213$	2.1213	$< 10^{-15}$
2	$3\sin(45)$	$\frac{3\sqrt{2}}{2} = 2.1213$	2.1213	$< 10^{-15}$
3	$2\cos(0)$	2	2.0	0
4	$2\sin(0)$	0	0.0	0
5	$2\cos(60)$	1	1.0	0
6	$2\sin(60)$	$\sqrt{3} = 1.7321$	1.7321	0
7	$2\cos(120)$	-1	-1.0	0
8	$2\sin(120)$	$\sqrt{3} = 1.7321$	1.7321	0
9	$2\cos(180)$	-2	-2.0	0
10	$2\sin(180)$	0	0.0	0
11	$2\cos(240)$	-1	-1.0	0
12	$2\sin(240)$	$-\sqrt{3} = -1.7321$	-1.7321	0
13	$2\cos(300)$	1	1.0	0
14	$2\sin(300)$	$-\sqrt{3} = -1.7321$	-1.7321	0
15	$\cos(30)$	$\frac{\sqrt{3}}{2} = 0.8660$	0.8660	$\sim 10^{-16}$
16	$\sin(30)$	$\frac{1}{2} = 0.5$	0.5	0
17	$2\cos(30)$	$\sqrt{3} = 1.7321$	1.7321	0
18	$2\sin(30)$	1	1.0	0

D.3 Difference Analysis

D.3.1 Main Differences

1. Symbolic results: SCME returns algebraic expressions such as $\frac{\sqrt{3}}{2}$, whereas GPT-4o-mini returns only the decimal value 0.8660254037844387.

2. Determinism: SCME yields consistent results across every call. GPT-4o-mini may produce different outputs due to the probabilistic nature of LLMs.

3. Performance: A batch of 18 computations took about 200 ms with the evaluated MCP version, compared with 8.49 s through the OpenAI API. The two durations are not like-for-like: the API time is dominated by sequential token generation,

whereas SCME runs a local function, so the figure shows that delegation adds no prohibitive delay, not that SCME is a faster reasoner. A re-measurement of the current engine version on a shared development host gave a median of about 0.8 s for a single call and about 1.1 s for a batch of 18.

4. Accuracy: For standard angles (30°, 45°, 60°), both produce equivalent results. However, for more complex computations (see Table 4.3 of the main volume), GPT-4o-mini frequently makes errors.

D.3.2 Transparency Notes

- Raw data: captured as a JSON log accompanying the project deliverables.
- Execution: scripted API call to the OpenAI endpoint.
- Model: OpenAI GPT-4o-mini (a weaker model, 128K context window).
- Credentials: the author's personal API key.
- Reproducibility: results can be reproduced by re-running the experiment under the same prompt and model version; each reported figure is a single run per condition.

D.3.3 Experimental Limitations

- GPT-4o-mini is a "weak model"; stronger models (GPT-4, Claude Opus 4.5) may yield better results on simple computations
- The experiment only tests trigonometry with standard angles; non-standard angles (17, 123) may produce different outcomes
- A single API call is insufficient to assess consistency; multiple calls are needed for verification

APPENDIX E
EVIDENCE ARCHIVE

The numbers in this capstone project come from recorded data. Each recorded set carries a code, E1 to E9, that the Source lines cite instead of a file name. The sets are kept by the author together with the project files; the source code of the evaluated engine release is public at the address given in the Introduction, and the other sets are available from the author on request.

Table E.1: Evidence items cited in the Source lines

Code	Content
E1	Results of the 54-case test run: case inputs, expected values, engine outputs and the performance block
E2	Responses and scores of the 54-case comparison for three models, without and with SCME
E3	Noise experiment: prompt sets, model answers and session screenshots
E4	Claude Opus 4.5 noise scores transcribed from the session screenshots
E5	Error analysis of the GSM8K-100 runs: gains and losses per run and loss classes
E6	Latency and concurrency measurements of the released server
E7	Token count of the tool description and input schema sent to a client
E8	Check of five finance calculations against an independent computation
E9	Metadata of the batch call compared with GPT-4o-mini
E10	Repeated-run experiment of GPT-4o-mini on the 54 cases in two conditions: raw answers, per-condition summary, failure labels and root-cause diagnosis

Source: compiled by the author.

APPENDIX F
APPLICATION PROPOSALS

F.0.1 Real-world applications of SCME

Illustrative example: plotting a power curve

A typical SCME use case is plotting graphs with irrational exponents. Consider plotting $y = x^{\sqrt{3}}$ at $x \in \{1.5, 2.0, 2.5, 3.0\}$. A bare LLM often errs on $\sqrt{3} \approx 1.732$ and the related powers; the screenshot of a related exponentiation error by a model without tool support now appears in Section 2.3.3 of the main volume.

The example shows how SCME handles graphics problems with irrational exponents. A bare LLM fails for three reasons: (1) miscomputing powers with irrational exponents such as $\sqrt{3}$; (2) rounding early rather than keeping high precision; (3) compounding errors across steps. SCME addresses all three failure modes with arbitrary-precision arithmetic.

Illustrative example: computing TikZ spacing

SCME computes spacing between elements in a TikZ diagram to ensure accurate symmetric alignment. The illustrative screenshots showing TikZ spacing calculation and WBS micro-task counting appear in the main volume.

F.0.2 Applications in Finance and Banking

At Ho Chi Minh University of Banking, this study pays special attention to SCME applications in finance and banking. AI agents are increasingly deployed in risk management, auditing, and investment research, whose pre-trade and post-trade analytics demand high numerical accuracy.

In foreign exchange (Forex) and securities analytics, pre-trade screening and post-trade review rely on technical analysis computed in batches over historical series. Computing indicators such as Moving Averages, the Relative Strength Index (RSI), and

Bollinger Bands requires high precision. Even a small error in these calculations can distort a valuation or a research conclusion, which can mislead a later human decision.

Figure 4.3 in the main volume shows the architecture of a batch analytics agent combining three components: the LLM for news sentiment analysis, RAG for historical data retrieval, and SCME for technical indicator computation.

In this architecture, the LLM Agent reads market sentiment from news. The RAG tool retrieves historical data from a vector database. The SCME MCP server computes technical indicators precisely. Results from the three components combine into pre-trade valuation and post-trade review reports that a human decision-maker reads, not into real-time trading decisions. SCME ensures the indicators stay correct, removing the risk of wrong signals from AI hallucination. This application targets offline analysis, research and low-frequency decision support, not high-frequency trading: the re-measurement in Section 4.1.4 of the main volume gave a median of about 0.8 s per call on a shared development host, because each call starts a new process, so latency-critical trading lies outside the scope of the artifact.

F.0.3 Deployment recommendations

For AI-developing organisations, the study recommends several actions. First, consider integrating SCME or similar dedicated computation tools into AI products. Second, apply the Delegated Computation model in high-accuracy fields such as finance, healthcare, and scientific research. Third, be transparent about the limits of AI systems on computation tasks.

For educational institutions, SCME opens many teaching opportunities. The tool can support effective math teaching and learning. Lecturers can design activities that help students detect AI hallucination and sharpen critical thinking. Exercises can blend traditional reasoning with tool-based verification.

For the developer community, the study stresses caution when using raw LLM calculations. The separation-of-concerns principle should be followed strictly. The LLM handles semantic reasoning, and dedicated tools handle computation. MCP or dedicated computation APIs are recommended for any critical calculation.

APPENDIX G
STATE OF RESEARCH: COMPARISON TABLES

Table G.1: Summary of representative international studies

Author	Year	Main contribution
Ji et al.	2023	Comprehensive survey of hallucination in NLG
Xu et al.	2024	Hallucination is inevitable for computable LLMs
Banerjee et al.	2024	Structural hallucination at every pipeline stage
Karpowicz	2025	Impossibility theorems for hallucination control
Dziri et al.	2023	Compositionality limits of transformers
Mirzadeh et al.	2024	GSM-Symbolic: fragility to changed numbers
Wang et al.	2024	Math-Shepherd: automatic process supervision
Wei et al.	2022	Chain-of-Thought (CoT) prompting
Schick et al.	2023	Toolformer – self-taught tool use for LLMs
Gao et al.	2023	PAL – program-aided reasoning

continued on the next page

Table G.1 (continued): Summary of representative international studies

Author	Year	Main contribution
Wang et al.	2023	MathCoder – code integration in math reasoning
Lewkowycz et al.	2022	Minerva – LLM for quantitative problems
Shen et al.	2024	Multi-agent LLMs that learn tool use
Yin et al.	2024	MuMath-Code – tool use plus data augmentation

Source: compiled by the author from the studies reviewed in this section: Banerjee, Agarwal, and Singla (2024); Dziri, Lu, Sclar, et al. (2023); Gao, Madaan, Zhou, et al. (2023); Ji et al. (2023); Karpowicz (2025); Lewkowycz, Andreassen, Dohan, et al. (2022); Mirzadeh et al. (2024); Schick, Dwivedi-Yu, Dessì, et al. (2023); Shen, Li, Chen, et al. (2024); K. Wang, Ren, Zhou, et al. (2023); P. Wang, Li, Shao, et al. (2024); Wei, Wang, Schuurmans, et al. (2022); Xu, Jain, and Kankanhalli (2024); Yin, You, Ji, et al. (2024).

Table G.2: Math performance of SOTA models (2024–2025)

Model	Benchmark	Accuracy	Note
o3 (OpenAI)	AIME 2024	91.6%	Reasoning model
Gemini 2.5 Pro	MATH	∼90%	Multimodal
DeepSeek-R1	GSM8K	∼92%	Open source
Claude Opus 4.5	54 custom	90.7%	This project
SCME	**SCME test set (54 cases)**	**100%**	**Tool-based**

Source: vendor-reported benchmark figures compiled by the author from Google DeepMind (2025); Guo et al. (2025); OpenAI (2025); the Claude Opus 4.5 and SCME rows are the author's experiment on the 54-case suite (evidence item E2).

G.1 LLM progress in mathematics (2023–2025)

This section presents the detailed review of LLM progress in mathematics that underlies the problem studied in Section 1.6 of the main volume.

The mathematical reasoning of large language models advanced sharply from 2023 to 2025, driven by larger model scale (over 100 billion parameters), Chain-of-Thought

prompting, and reinforcement learning. This review covers sources available up to 2025; models and benchmarks released later are not covered, and the design of SCME does not depend on any particular model release.

On milestones, 2023 marked the maturation of CoT prompting, which improved results on the GSM8K benchmark by 28–62% (Cobbe, Kosaraju, Bavarian, et al., 2021). In 2024, OpenAI launched the o1/o3 reasoning models with test-time compute scaling; these models reached top-500 level on the AIME 2024 U.S. math olympiad. In early 2025, leading models such as Gemini 2.5 Pro and DeepSeek-V3 reported performance at or above human-expert level on several quantitative benchmarks.

Leading AI companies now ship reasoning models, calculator tools, and hybrid methods. OpenAI's o1/o3 family (OpenAI, 2025) (including o1-mini and o3-mini) trains step-by-step reasoning with reinforcement learning and extended inference time, and its Code Interpreter computes exactly, independent of token prediction (as of October 2026). Google's Gemini 2.0 Flash and 2.5 Pro combine Chain-of-Thought with step-level decomposition and score strongly on AIME 2025. Anthropic focuses on reinforcement learning with verifiable rewards and Constitutional AI oversight. Meta's Llama 3 supports tool usage natively (Grattafiori et al., 2024), DeepSeek-R1 incentivizes reasoning through reinforcement learning alone (Guo et al., 2025), and DisCIPL (Grand et al., 2025) coordinates smaller models with performance comparable to o1 and GPT-4o.

Open-weight models. Llama 3 rivals leading proprietary models and releases its largest model publicly (Grattafiori et al., 2024), Mistral 7B ships under the Apache 2.0 licence (Jiang et al., 2023), and the Qwen2.5 flagship reportedly matches Llama-3-405B-Instruct (Yang et al., 2024). SCME needs only an MCP client with tool calling, so it applies to any such model, but none was evaluated: the experiments use GPT-4o-mini, GPT-5-mini, Claude 3.5 Haiku, and Claude Opus 4.5, and whether their with-and-without-tool gap matches is untested.

On benchmark performance, leading models now approach perfect scores on standard benchmarks. On GSM8K (primary-school math), CoT yields a 28–62% improvement. On MATH (competition level), models reach roughly 80–90% through reinforcement-learning reasoning. On a new 2025 benchmark of 77 PhD-level proofs, however, completion rates still fall short.

Table G.2 summarizes vendor-reported math performance of SOTA models for 2024–2025 (benchmarks GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021)); the SCME row reports this project's 54 custom cases.

On remaining challenges, despite vendor-reported accuracies of about 90% on public math benchmarks (Table G.2), hallucinations still arise in complex computation, especially: (1) PhD-level proofs and abstract problems; (2) novel problems that require symbolic manipulation or multi-step logic without a prepared formalization; and (3) high inference cost when using frontier models. Hybrid methods that combine reinforcement learning with symbolic processing or that coordinate smaller models show promise but do not fully solve the problem.

Benchmark scores and robustness. High benchmark scores alone do not show that a model computes rather than recalls. Mirzadeh et al. (2024) built GSM-Symbolic from symbolic templates, so names and numbers vary while the logic stays fixed. Accuracy differs across instantiations of the same question, drops when only the numbers change, declines as clauses grow, and one irrelevant clause can cut accuracy by up to 65% across state-of-the-art models, suggesting that LLMs replicate trained reasoning steps rather than reason genuinely. This project tests a different perturbation, padding the same question with long context (Chapter 2); both lines support one conclusion: accuracy on a clean, familiar item overstates reliability on a changed item.

Why 95% is still not enough. In critical fields such as finance, healthcare, and engineering, a 5% error can cause severe harm. A 5% error in drug dosage can endanger a patient; a flawed financial calculation can lose millions. This is why delegating computation to a dedicated tool such as SCME retains its value: it can achieve near-perfect accuracy that does not depend on any specific model's reasoning.

APPENDIX H
IDE INTEGRATION

The architecture diagram of SCME integration with IDEs appears in Chapter 3 of the main volume.

Integration test results on each platform appear in Table H.1.

Table H.1: IDE integration test results

Criterion	VSCode	Cursor	Claude Code
MCP server starts	✓	✓	✓
Tool is recognised	✓	✓	✓
Computation correct	✓	✓	✓

Source: author's manual integration checks of the SCME MCP server in VSCode, Cursor and Claude Code.

APPENDIX I
IEEE 754 NUMBER REPRESENTATION DETAILS

This appendix provides the IEEE 754 details that underlie the root-cause discussion in Section 2.4.3 of the main volume. The IEEE 754 standard specifies binary floating-point representation with three components: sign bit, biased exponent, and mantissa. Single precision uses 32 bits with a 23-bit mantissa; double precision uses 64 bits with a 52-bit mantissa.

Because the number of bits is limited, computers can store only binary rational numbers exactly. Common decimal numbers such as 0.1 must be approximated, producing a well-known result: $0.1 + 0.2 \neq 0.3$ in most programming languages. Specifically, 0.1 in decimal equals the infinite repeating binary string 0.000110011..., which rounding turns into a small error. This error accumulates across chained calculations and becomes especially serious in multi-step problems. This is one of the fundamental sources of calculation errors in computers generally and in AI systems specifically.

To address this issue, specialized calculation tools typically use epsilon comparison (ε). Instead of comparing two real numbers directly, the system checks whether their difference is smaller than an allowed threshold. The evaluated SCME release (v1.1.0) does not apply an epsilon: it converts the symbolic result to a double-precision number and also returns the symbolic form. A revision of January 2026 briefly applied the threshold $\varepsilon = 10^{-5}$ to clean IEEE 754 artifacts such as $0.1 + 0.2$, and the current development version replaced it by rounding to a requested number of decimal places (15 by default); the threshold constant is still defined in the configuration file but is no longer used. The design trade-off remains visible: a result returned as a double-precision number cannot carry more than about 15 to 17 significant digits, so computations that need more lie outside the scope of the artifact and are listed as a limitation.

APPENDIX J
SCOPE, BASELINES AND MEASUREMENT LIMITS

This appendix presents the scope, baselines and measurement limits of the study detailed in Section 4.1.6 of the main volume.

J.0.1 Scope, baselines and measurement limits

Summary of limits. This paragraph gathers every adverse result; each is reported in full where noted. The evaluated version still leaks the engine's absolute path (14 of 15 probes before the patch), and the optional visualization hook executed two of three Python-syntax probes on the host (Section 4.1.4 of the main volume). On GSM8K-100 the tool runs scored 12, 15 and 21 points below the baseline; no expert panel reviewed the instruments; MATH was not run (Section 4.1.5 of the main volume). In the repeated runs below, the planned run scored B 0.4037 against A 0.6704, and the clean replication B 0.6926 is not above A within its interval 0.6351 to 0.7446. No user study was run. The other models ran once per condition. The 54 cases are author-constructed. Open-weight models were not evaluated.

This subsection states, in the form a design-science evaluation requires, what the artifact does not claim.

Baselines and versions. The 54-case comparison uses Claude Opus 4.5 (the frontier tier among these models), Claude 3.5 Haiku and GPT-4o-mini. GPT-5-mini appears only in the noise suite, its built-in code execution disclosed there. Claims bind to named versions because versions drift; the target is the delegation effect, not a leaderboard rank. Open-weight models (Llama, Mistral, Qwen) were not evaluated; a registry for the second study was designed but not executed.

Operations. The evaluated version implements 27 operations (the maintained code has 30): symbolic equation solving, differentiation, integration, limits, statistics, regres-

sion, distribution tests, combinatorics, number theory, and matrix and vector algebra on SymPy, SciPy and NumPy. Partial differential equations and constrained or complex linear optimisation are out of scope; the next design iteration would add them, testing with the same error probes. Library delegation itself is not claimed as an academic contribution; the claimed contributions are the five design principles, the 54-case evaluation protocol, and the engine and benchmark released under the MIT licence. Five finance calculations (compound interest, straight-line depreciation, declining-balance book value, net present value, internal rate of return) ran through the released engine as a management information systems scope check. All five used the existing evaluation and polynomial-solving operations, with no dedicated finance operation, and matched an independent NumPy and SciPy computation exactly for the first four and within 3.1×10^{-17} for the internal rate of return (evidence item E8). The model still composes each formula itself; the engine does not check that composition.

Scoring and suites. The 54 cases were scored by manual comparison with a fixed answer key; an automatic pipeline recomputing gold answers with a verified engine served the 2,300 machine-scored Episode A calls; re-scoring the 54 cases with it is the cheapest check of manual scoring. Three limits belong together: author-constructed problems and manual scoring; each model ran once per condition, except GPT-4o-mini, which ran five times (next paragraph). This is acceptable for a formative episode (Venable, Pries-Heje, & Baskerville, 2016); the limits motivated the machine-scored episodes of Section 4.1.5 of the main volume. The 50-problem noise suite is separate, not a subset, and the two are not pooled. Repeated runs of a subsample are scheduled with the planned think-aloud study on the same 54 cases, so run-to-run variance can then be reported.

Repeated runs with the model choosing the tool. On 7 October 2026, GPT-4o-mini ran the 54 cases five times per condition (270 answers each). Alone (condition A) it scored 181 of 270 (0.6704; runs 0.6667 to 0.6852). Calling the tool itself (condition B) it scored 109 of 270 (0.4037; runs 0.3704 to 0.4444), an adverse result for unaided tool use with the evaluated schema. Of 161 wrong answers, 119 had no usable tool result (114 exits with status 1, 5 'Error' answers); 42 used the wrong operation. Of the 114 exits: 84 guessed parameter names (the schema lists no per-operation names); 16 engine-

copy defects (pint missing from the forced virtual environment; one ’factorial’ call the readme documents but the code rejects); 4 invalid plans; 10 unresolved. Excluding the 119 rows, B answered 109 of 151 (0.7219), a conditional figure, not a full-run result. A first rerun of the 119 failures answered 25 correctly, and a second rerun of the 81 still failing, with libraries present and value hints added, answered 55; both mixed tool text and environments, so they stay exploratory. A clean five-run replication with one corrected tool text (parameter names and value vocabularies) and a full environment from the first call answered 187 of 270 (0.6926; per run 0.6667 to 0.7037, range 0.037; 95 percent interval 0.6351 to 0.7446). The gain over the planned run is 0.2889. Against condition A (0.6704) the difference is 0.0222, small compared with the interval width, so the data do not show that the tool beats the model alone on these 54 cases; they show the earlier deficit came from the tool text and the environment. Seventeen tool errors remained (combinatorics 10, integrate 5, batch 1, `stats_regress` 1). The model chose the expected operation in 207 of 270 answers (0.7667).

Malformed input and tool selection. Malformed JSON cannot crash the artifact: handlers turn a decoding failure into a structured error, and the 15 probes of Section 4.1.4 of the main volume produced no crash. Retries on malformed model output belong to the agent loop; the Episode B runner retries up to three times on an unparseable plan. A built-in fallback is future work. Wrong tool choice is now measured for GPT-4o-mini (previous paragraph); the result supports design principle DP3, a fault-tolerant interface, and the next iteration: per-operation parameter schema, argument aliases, repair of the engine environment; the fix was then run as the clean five-run replication reported above; what remains are the wrong-operation choices (63 of 270 answers did not match the expected operation) and the remaining tool errors in combinatorics and integration.

Why an external tool when models can execute code. Some models run code themselves: GPT-5-mini kept 100% accuracy at low noise with built-in execution, so built-in execution can deliver much of SCME’s effect; this project does not claim to outperform it. The case for an external tool rests on unmeasured design properties: computation stays under the organisation’s control, so accounting or audit figures need not reach a vendor sandbox; the fixed operation set and structured error contract work with any model and

MCP client, including local models offline; and the operation list is closed and reviewable, unlike generated code. These are design arguments to be tested, not results.

Users. The capstone evaluates the artifact technically (accuracy with and without SCME, latency, error probes). No human user study was run, so usability, trust and productivity effects are unmeasured. Formative technical evaluation precedes summative practitioner use.

APPENDIX K
COST–BENEFIT ANALYSIS

This appendix presents the cost-benefit analysis of Section 4.2.3 of the main volume together with its full data table.

K.0.1 Cost–benefit analysis

Table K.1 compares the cost and benefit of SCME against advanced reasoning models such as o3 and Gemini 2.5 Pro.

The comparison supports a division of labour rather than a verdict, because no head-to-head comparison was run: 100% is the result on the project's own 54-case suite of pre-formatted expressions, whereas the accuracy, cost and response-time cells for the reasoning models in Table K.1 are qualitative and were not measured in this project. A like-for-like comparison, running the reasoning models on the same 54 cases or SCME behind a model on a public benchmark subset, is left to future work. On response time no reasoning-model latency was measured, so no speedup factor is claimed; the measured latency of the released server, about 0.86 s median end-to-end for sequential calls and 2.4 s with eight concurrent callers, is reported in Section 4.1.4 of the main volume. The schema overhead was measured by tokenising the tool list that the released server sends to a client: the single calculate tool, with its description and input schema, comes to 887 tokens with the cl100k_base encoding and 889 with o200k_base (evidence item E7), and this amount is added to every request in which the tool is offered. At an input price of USD 0.15 per million tokens (the GPT-4o-mini price recorded in the project data), 1,000 requests add about USD 0.13, so the overhead is small in money but not zero in context length. Yet reasoning models such as o3 handle open-ended, abstract problems that SCME cannot. The best solution combines both. Use SCME for calculations that demand exact accuracy. Use reasoning models for tasks requiring creativity and abstract thought. Application proposals are given in Appendix F.

Table K.1: Cost–benefit comparison: SCME vs advanced reasoning models

Criterion	**SCME**	**Reasoning models (o3, Gemini 2.5 Pro)**	**Note**
Accuracy (54-case suite)	100% (measured)	Reported high on public benchmarks; not measured in this project	No head-to-head comparison run
API cost	No per-call API fee; self-hosting, maintenance and about 890 schema tokens per prompt (measured)	Per-token vendor pricing; not measured in this project	Cost moves from per-call fees to hosting and prompt overhead
Response time	0.86 s median end-to-end for sequential calls; 2.4 s with eight concurrent callers (measured)	Slower on reasoning prompts; not measured in this project	Direct speed comparison not run
Infrastructure	Local/MCP	Cloud API	SCME flexible
Integration complexity	Medium	Low	Comparable
Scope	27 operations	Unlimited	o3/Gemini broader

Source: SCME values measured by the author: accuracy on the 54-case suite (evidence item E1), latency medians (evidence item E6) and schema tokens (evidence item E7); the operation count is the declared scope of the evaluated version; the reasoning-model cells are qualitative judgements that are not measured in this project; the remaining cells are the author's qualitative labels.

APPENDIX L
RECOMMENDATIONS

This appendix presents the application recommendations developed in Chapter 4 of the main volume.

L.0.1 Recommendations

Application Recommendations

AI development organizations should consider integrating SCME or similar tools into their products. The Delegated Computation model should be applied in fields that require high accuracy. Organizations should also be transparent about the capabilities and limitations of AI in computation.

For educational institutions, SCME can serve as an effective tool to support mathematics teaching. Instructors should guide students to recognize AI hallucination and to develop critical thinking. For developers, the principle of separation of concerns should be followed strictly. LLMs should focus on reasoning, while computation should be delegated to specialized tools through MCP or an API.

Future Development Directions

In the short term (6 to 12 months), the system should add operations for ordinary differential equations, optimization, and sparse matrices. Integrating a caching layer and expanding multi-LLM support are also priority goals.

In the medium term (1 to 2 years), a Vietnamese natural-language interface and visual mathematics support through OCR will be developed. Formal verification methods will also be studied for application.

In the long term (beyond 2 years), the project aims to build an evaluation benchmark and a research platform for hybrid AI systems. Domain-specific variants are also part of the roadmap.

Vision: SCME in the Agentic AI Ecosystem

The trend in AI development is shifting strongly from single models to agentic AI systems. These systems consist of multiple agents coordinated by an orchestrator. As a working hypothesis, SCME could become a standard computation component for AI agents with numerical needs if three conditions hold: adoption of the MCP protocol continues, the artifact is hardened and re-evaluated on public benchmarks beyond the 54-case suite, and independent users replicate the accuracy and safety findings. This capstone project supports the hypothesis only for the evaluated models and the released suites.

Instead of each agent having to learn to compute on its own, agents can delegate fully to a specialized computation agent such as SCME. This workflow follows the principle that the LLM selects the right tool and the execution framework calls it. With the growing adoption of the MCP protocol, a wide range of AI systems can achieve reliable numerical accuracy through delegation. Whether the same benefit extends from frontier models to smaller models is an open hypothesis; this study evaluated only Claude and GPT models, so no claim is made for untested model classes.

Overall, the results suggest that SCME is a feasible and effective solution for mitigating AI hallucination in mathematical calculation. They also open further opportunities for research and application in hybrid AI systems.

A related benchmark study by the author and the advisor has been accepted for publication in the Asian Journal of Data Science and Artificial Intelligence and is forthcoming (see the List of Publications).

APPENDIX M
VISUAL ILLUSTRATIONS OF CALCULATION ERRORS

This appendix illustrates the calculation errors discussed in Section 2.3.3 of the main volume with three screenshot figures and the accompanying text.

M.0.1 Visual Illustrations of Calculation Errors

The Claude exponentiation screenshot (a typical exponentiation error without tool support, corresponding to Case Study 2 above) has moved to Section 2.3.3 of the main volume.

The screenshots of the 50-question random test with Claude Opus 4.5 and the AI counting error (claiming three pages when only two exist) appear in Figure 2.3 in Section 2.3.3 of the main volume.

These examples show that calculation hallucination occurs not only in complex calculations but also in simple tasks such as counting.

APPENDIX N
ARCHITECTURE OF LARGE LANGUAGE MODELS

This appendix describes the architecture of large language models that underlies the problem studied in Section 1.2 of the main volume.

N.0.1 Architecture of large language models (LLM)

A large language model (LLM) is a deep-learning model with billions of parameters, trained on large text corpora to perform natural-language tasks.

Transformer architecture

Modern LLMs rest on the Transformer architecture (Vaswani, Shazeer, Parmar, et al., 2017), introduced in the 2017 paper "Attention Is All You Need". This architecture reshaped natural-language processing by removing traditional sequential and convolutional mechanisms such as recurrent neural networks.

Self-attention lets the model measure how strongly each position in the input sequence relates to every other when processing each token. Multi-head self-attention runs several attention heads in parallel to capture different relational patterns. Positional encoding adds position information. Feed-forward networks, layer normalization, and residual connections stabilize and speed up training.

The overall architecture diagram of the Transformer model appears in Chapter 1 of the main volume.

LLM training process

LLM training has two main stages. Before training, text must be split into smaller units called *tokens* through tokenization. A token can be a whole word, a sub-word, or even a single character, depending on the tokenization algorithm (BPE, WordPiece, or SentencePiece).

Stage one is pre-training on large text corpora such as the web, books, and source code. The core principle is next-token prediction over a probability distribution: the model learns to estimate the probability of each next token given the preceding tokens. This runs as self-supervised learning and demands large compute resources. Stage two fine-tunes and aligns behavior. Methods include supervised fine-tuning (SFT) on high-quality instruction data and reinforcement learning from human feedback (RLHF) to align behavior with user preferences.

Table N.1 compares the size of representative LLMs (parameter counts appear only where the provider has published them):

Table N.1: Size of representative LLMs

Model	**Parameters**	**Year**	**Notes**
GPT-3	175B	2020	Few-shot learning
GPT-4	Not disclosed	2023	Multimodal, MoE
Claude 3 Opus	Not disclosed	2024	Constitutional AI
Llama 3	70B	2024	Open source

Source: compiled by the author from Anthropic (2024); Brown, Mann, Ryder, et al. (2020); Grattafiori et al. (2024); OpenAI (2023); providers have not published the parameter counts of GPT-4 and Claude 3 Opus (status in October 2026), so no figure is given for them.

APPENDIX O
HALLUCINATION IN MATHEMATICAL COMPUTATION

This appendix examines hallucination in mathematical computation, the failure mode introduced in Section 1.3 of the main volume.

O.0.1 Hallucination in mathematical computation

Mathematics is especially sensitive to hallucination because answers are binary (right or wrong).

The root cause lies in the LLM principle of predicting tokens by probability rather than understanding concepts. Training data may also contain outdated or biased information. Architectural limits such as a fixed context window and a lack of long-term memory also contribute to errors.

Specific computational errors often arise because LLMs treat numbers as character strings rather than numeric values. The system also lacks post-generation verification and often compares strings instead of values (for example, the "9.11 > 9.9" error).

The first mechanism is how numbers are split into tokens. Subword tokenizers compress natural text, not place value. In tiktoken's `cl100k_base` encoding, 1234567 becomes 123, 456, and 7, cut from the left and misaligned with units, thousands, and millions, while 3.14159 becomes four tokens (3, a decimal point, 141, and 59). The same encoding splits 9.11 and 9.9 into integer part, decimal point, and fractional part (11 and 9), so the fractions reach the model as the integers 11 and 9. This matches the "9.11 > 9.9" error above, though tokenization is not the only cause. Training-side remedies would require pre-training a new model, so they do not help an application using an existing model.

The second mechanism is error accumulation over many steps. Dziri et al. (2023) describe multi-step tasks as computation graphs and show that transformer LLMs solve them by matching linearized subgraphs seen in training, with performance decaying as

the graph becomes deeper. A long financial or engineering calculation is exactly such a graph, so a small per-step error rate compounds along the chain.

A third mechanism concerns what attention can compute. Hahn (2020) proves that self-attention cannot model periodic finite-state languages or hierarchical structure unless layers or heads grow with input length; this is a theoretical limit, not a measure of arithmetic accuracy. Nogueira, Jiang, and Lin (2021) show that a number's surface form shapes how well models learn addition: with subword number pieces they fail at five-digit addition, with position tokens they learn it up to 60 digits, and no setting yields rules independent of training length. With tokenization and error accumulation, this explains why multi-step arithmetic stays fragile when the language is fluent, and why exactness is better delegated than trained.

The consequences include financial loss in transaction computation, distorted conclusions in scientific research, and safety risks in medicine or engineering. This is the main motivation for developing SCME, which lets an LLM delegate computation to a trusted tool.

APPENDIX P
BACKGROUND TABLES

This appendix collects background tables and background discussion that were moved out of the main volume so that the main volume stays focused on the argument. Each item keeps its content unchanged; only pointer text was adapted.

P.1 Representative LLMs (2023 to 2024)

Section 1.1 of the main volume describes the LLM market of 2023 to 2024; Table P.1 compares representative models of that period.

Table P.1: Comparison of representative LLMs (2023 to 2024)

Model	Developer	Year	Key features
GPT-4	OpenAI	2023	Multimodal, large scale
Claude 3	Anthropic	2024	Safe, deep dialogue
Gemini	Google	2023	Google Search integration
Llama 3	Meta	2024	Open source

Source: compiled by the author from Anthropic (2024); Google (2023); Grattafiori et al. (2024); OpenAI (2023).

P.2 LLM market milestones, 2020 to 2026

The following paragraphs were moved here from Section 1.1 of the main volume to keep the main text within its page budget.

The 2020 to 2022 period marked the maturation of language-modeling platforms. In 2020, OpenAI released GPT-3 with 175 billion parameters and demonstrated strong few-shot learning (Brown et al., 2020). This model clarified the critical role of scale in LLM capability.

The 2023 to 2024 period brought competition and diversification to the LLM market. Major vendors released new versions to improve dialogue and context retention. An-

thropic shipped Claude 2 and Claude 3 (Anthropic, 2024). Google introduced Gemini with multimodal capability (Google, 2023), while Meta pursued open source by releasing Llama 2 and Llama 3. These moves reflect growing diversity in models and development strategies.

The 2022–2023 period brought GenAI into mainstream use. In November 2022, ChatGPT launched and reached 100 million users within two months, proving the technology's accessibility for general users. OpenAI released GPT-4 in March 2023 with clear gains in natural language processing (OpenAI, 2023). The model scaled up, handled complex context better, and processed both text and images. Fine-tuning further improved performance on specialized tasks.

The 2024 to 2026 period brought agent infrastructure and reasoning models, including the Model Context Protocol (November 2024) and the frontier models tested in this project (October 2026).

P.3 Chapter 3 summary (recap moved from the main volume)

Chapter 3 presented the SCME solution in full. On method, the study applied Design Science Research (DSR) with six stages and a test suite of 54 questions across 12 categories. The system design uses a four-layer architecture with clear separation of concerns. It supports 27 operations from basic arithmetic to advanced calculus. Deployment achieves high efficiency: startup under 500ms and memory under 50MB.

Evaluation shows SCME reaching 100% accuracy in the 24 December 2025 run on both basic and advanced tests. It lifts average LLM performance by 17.9% (from 82.1% to 100%) and integrates smoothly with VSCode, Cursor, and Claude Code. The qualitative comparison suggests an advantage of SCME over CoT, RAG, and PAL on computational accuracy for this test set; no head-to-head experiment was run. Finally, short-, medium-, and long-term proposals set directions for both research and application.

On the design side, the project proposes a four-layer architecture to ensure modularity and scalability. The first layer is the AI Application, which includes Claude Desktop, Cursor, and VSCode. The second layer is the MCP Server, written in TypeScript, responsible for validation and routing. The third layer wraps the Python CLI to handle communication through the JSON standard. The final layer is the Python Computation Core, which uses strong libraries such as SymPy, NumPy, SciPy, and mpmath.

On the implementation side, the system supports 27 operations across seven categories: Basic, Statistics, Regression, Complex Numbers, Units, Advanced Mathematics, and Calculus. Performance remains strong: startup time stays under 500 ms, computation time under 50 ms, and memory use below 50 MB. These figures indicate that the system is lightweight and efficient.

P.4 Reliability methods: background discussion

Section 1.4 of the main volume summarizes the methods for ensuring computational reliability and their comparison; the fuller background discussion removed from the main volume is kept here.

Having covered MCP as a standard protocol for connecting AI with external tools, this section analyzes different *strategies* for ensuring computational reliability. Two concepts must be separated: MCP is the technical infrastructure (how to connect), while the methods below are strategic approaches (how to solve the problem).

The relationship is as follows. MCP provides the “pipe” through which AI talks to tools, while the methods decide “how to use that pipe” or “whether to use it at all”. In particular, Tool Use (Section 1.4.3 of the main volume) is the strategy that uses MCP infrastructure directly, and it is also the foundation on which SCME is built. Other methods such as CoT or PAL can either combine with MCP or work independently.

To ensure reliability in high-precision tasks, researchers have proposed many methods. This section analyzes four main methods and compares their strengths and weaknesses.

APPENDIX Q
ROOT CAUSES OF LLM CALCULATION ERRORS

This appendix gives the full Five Whys analysis and the summary of the causes of LLM calculation errors that support Section 2.4 of the main volume.

Q.1 Five Whys Analysis

Root problem: LLMs calculate incorrectly.

The "Five Whys" method identifies the root cause. First, LLMs calculate incorrectly because the system relies on probabilistic prediction rather than actually "computing" (Dziri et al., 2023; Mirzadeh et al., 2024). Second, probabilistic prediction produces errors because the model learns patterns from text instead of learning arithmetic. Third, pattern learning falls short because math requires precise logic and does not accept "approximations". Fourth, LLMs lack precise logic because the Transformer architecture was not designed for arithmetic. Finally, keeping the architecture unchanged is a design trade-off: LLMs optimize for language rather than math.

Q.2 Summary of Causes

Table Q.1 summarizes the causes of LLM calculation errors.

Table Q.1: Summary of LLM calculation error causes

Group	**Cause**	**Consequence**
Architecture	Transformer not designed for arithmetic	Basic errors (+, -, ×, ÷)
Representation	Numbers processed as character strings	Comparison errors (9.11 > 9.9)
Training	Learning from text, not formal mathematics	Lack of rigorous reasoning
Floating point	IEEE 754 precision limits	Error accumulation
Multi-step	Each step has error probability	Compounded errors

Source: compiled by the author from Dziri et al. (2023); Goldberg (1991); Mirzadeh et al. (2024).

REFERENCES

Anthropic. (2024). *Claude 3 model card.* Retrieved 2024-12-23, from https://anthropic.com/claude

Banerjee, S., Agarwal, A., & Singla, S. (2024). *LLMs will always hallucinate, and we need to live with this.* arXiv preprint. doi: 10.48550/arXiv.2409.05746

Brown, T., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*, 1877–1901.

Cobbe, K., Kosaraju, V., Bavarian, M., et al. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Dziri, N., Lu, X., Sclar, M., et al. (2023). Faith and fate: Limits of transformers on compositionality. In *Advances in neural information processing systems* (Vol. 36). doi: 10.48550/arXiv.2305.18654

Gao, L., Madaan, A., Zhou, S., et al. (2023). PAL: Program-aided language models. In *International conference on machine learning (icml)* (pp. 10764–10799).

Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, *23*(1), 5–48. doi: 10.1145/103162.103163

Google. (2023). *Gemini: A family of highly capable multimodal models.* Retrieved 2024-12-23, from https://deepmind.google/technologies/gemini/

Google DeepMind. (2025). *Gemini 2.5: Our newest gemini model with thinking.* Retrieved 2025-12-24, from https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/ (Deep Think mode with code execution, AIME 2025: 86.7%)

Grattafiori, A., et al. (2024). *The Llama 3 herd of models.* arXiv preprint arXiv:2407.21783. doi: 10.48550/arXiv.2407.21783

Guo, D., et al. (2025). DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, *645*, 633–638. doi: 10.1038/s41586-025-09422-z

Hahn, M. (2020). Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, *8*, 156–171. doi: 10.1162/tacl_a_00306

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., … Steinhardt, J. (2021). Measuring mathematical problem solving with the MATH dataset. In *Advances in neural information processing systems (neurips) datasets and benchmarks track.* (arXiv:2103.03874)

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., … Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, *55*(12), 1–38. doi: 10.1145/3571730

Jiang, A. Q., et al. (2023). *Mistral 7B.* arXiv preprint arXiv:2310.06825. doi: 10.48550/arXiv.2310.06825

Karpowicz, M. P. (2025). *On the fundamental impossibility of hallucination control in large language models.* arXiv preprint. doi: 10.48550/arXiv.2506.06382

Lewkowycz, A., Andreassen, A., Dohan, D., et al. (2022). Solving quantitative reasoning

problems with language models. *Advances in Neural Information Processing Systems*, *35*, 3843–3857.

Mirzadeh, I., Alizadeh, K., Shahrokhi, H., Tuzel, O., Bengio, S., & Farajtabar, M. (2024). *GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models.* arXiv preprint. doi: 10.48550/arXiv.2410.05229

Nogueira, R., Jiang, Z., & Lin, J. (2021). *Investigating the limitations of transformers with simple arithmetic tasks.* arXiv preprint arXiv:2102.13019. doi: 10.48550/arXiv.2102.13019

OpenAI. (2023). *Gpt-4 technical report.* Retrieved 2024-12-23, from https://openai.com/research/gpt-4

OpenAI. (2025). *Introducing o3 and o4-mini.* Retrieved 2025-12-24, from https://openai.com/ (Code Interpreter integrated into chain-of-thought reasoning)

Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, *36*.

Shen, W., Li, C., Chen, H., et al. (2024). Small LLMs are weak tool learners: A multi-LLM agent. *arXiv preprint arXiv:2401.07324*.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*.

Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems*, *25*(1), 77–89. doi: 10.1057/ejis.2014.36

Wang, K., Ren, H., Zhou, A., et al. (2023). MathCoder: Seamless code integration in LLMs for enhanced mathematical reasoning. *arXiv preprint arXiv:2310.03731*.

Wang, P., Li, L., Shao, Z., et al. (2024). Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*. doi: 10.18653/v1/2024.acl-long.510

Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, *35*, 24824–24837.

Xu, Z., Jain, S., & Kankanhalli, M. (2024). *Hallucination is inevitable: An innate limitation of large language models.* arXiv preprint. doi: 10.48550/arXiv.2401.11817

Yang, A., et al. (2024). *Qwen2.5 technical report.* arXiv preprint arXiv:2412.15115. doi: 10.48550/arXiv.2412.15115

Yin, S., You, W., Ji, Z., et al. (2024). MuMath-Code: Combining tool-use large language models with multi-perspective data augmentation for mathematical reasoning. *arXiv preprint arXiv:2405.07551*.